λ_S^0 — A Formal Calculus for Straight-Line Typestate Checking

Alexandre Zua Caldeira
Independent Researcher
alex@zuacaldeira.com

March 2026

Abstract

Session types discipline the order in which methods may be called on an object, but existing tools for session types on objects—Mungo, Bica Reborn—lack a published formal term-language calculus grounding their type-checking algorithms. We present λ_S^0, the simplest layer of a stratified calculus family for session-typed objects. λ_S^0 formalises straight-line client code: classes, fields, **new**, and method calls—no conditionals, loops, or concurrency. Session types in this fragment are restricted to Branch and End (no selection, recursion, or parallel), so state spaces are finite trees and each client follows exactly one root-to-leaf path. We give syntax, a pre/post typing judgement with five rules, a small-step operational semantics with four reduction rules, and prove type soundness (preservation, progress, session completion). Every typing rule has a direct, documented correspondence to a line in the Bica Reborn `TypestateChecker` implementation. Worked examples on a `FileHandle` protocol show how typing derivations trace paths in the session-type Hasse diagram.

1 Introduction

Session types [9] ensure that objects are used according to a prescribed protocol—a sequence of method calls respecting branching structure, selection, and recursion. When applied to object-oriented languages, session types become *typestates* [13]: the type of an object changes as methods are invoked, and the type system enforces that only protocol-permitted methods are called in each state.

Several tools enforce session types on Java-like objects—Mungo [11] and its successors, the Plaid language [1], and our own Bica Reborn [7]. Despite their practical maturity, none of these tools has published a formal term-language calculus that precisely specifies the typing rules and operational semantics of the client-side code they check. The typing algorithms are described informally or by reference to implementation code, making it impossible to state and prove soundness.

This paper fills that gap for the simplest case. We define λ_S^0, a calculus for *straight-line client code*: the client allocates session-typed objects with **new**, calls methods on them, sequences operations with **let**, and must drive every object to its terminal state. There are no conditionals (λ_S^2 adds those), no loops (λ_S^3), no inheritance (λ_S^4), and no concurrency (λ_S^6).

The key restriction of λ_S^0 is that the client cannot observe the outcome of an internal choice (selection). Consequently, session types in this fragment use only *Branch* and *End*—no Select, no Rec, no Parallel. State spaces are finite trees with a unique top (initial state) and unique bottom (terminal state), and the well-typed client follows exactly one root-to-leaf path.

Contributions.

1. A formal syntax for λ_S^0 (Section 3): a minimal Java-like subset with **new**, method call, **let**, and **skip**.

2. A pre/post type system with five rules (Section 4): $\mathcal{CT}; \Gamma \vdash e : T \dashv \Gamma'$, where session state advances through the typing environment. Each rule is annotated with its correspondence to a specific line in Bica Reborn's `TypestateChecker.java`.

3. A small-step operational semantics with four reduction rules (Section 5), where getting stuck corresponds to a protocol violation.

4. Type soundness (Section 6): preservation, progress, and session completion, proved by structural induction with full proof sketches.

5. Worked examples (Section 7) showing correct and rejected clients for a `FileHandle` protocol, with typing derivations that trace paths in the Hasse diagram.

Outline. Section 2 recaps session types restricted to the λ_S^0 fragment. Section 3 defines the term language. Section 4 presents the type system. Section 5 gives the operational semantics. Section 6 proves type soundness. Section 7 walks through worked examples. Section 8 discusses the correspondence with Bica Reborn and the roadmap to richer fragments. Section 9 concludes.

2 Session Types (Recap)

We briefly recall the session type framework as developed in Steps 1 [3] and 5 [2], restricted to the fragment needed for λ_S^0.

2.1 Grammar

The full session type grammar supports branch, selection, parallel, recursion, and end. For λ_S^0, we restrict to:

Definition 2.1 (λ_S^0 Session Types)**.**

$$S \quad ::= \quad \mathbf{end} \quad | \quad \&\{m_1 : S_1, \ldots, m_n : S_n\}$$

where $n \geq 1$ and the method names m_i are pairwise distinct.

This excludes $\oplus$ (selection), $\mu X.S$ (recursion), $S_1 \parallel S_2$ (parallel), and type variables X. The restriction is sound for λ_S^0 because without conditionals the client cannot observe internal choices, without loops it cannot exploit recursion, and without threading it cannot use parallelism.

Example 2.2 (FileHandle)**.** The `FileHandle` protocol [7]:

$$S_{\mathrm{FH}} \;=\; \&\{\mathit{open} : \&\{\mathit{read} : \&\{\mathit{close} : \mathbf{end}\},\ \mathit{write} : \&\{\mathit{close} : \mathbf{end}\}\}\}$$

2.2 State Space

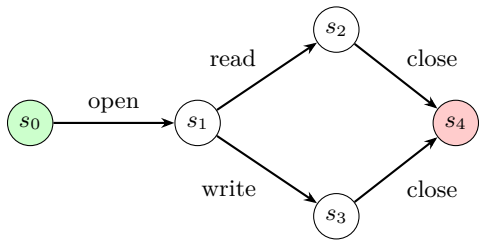
Given a session type S, the state-space construction of [3] produces a labelled transition system $\mathcal{L}(S) = (Q, q_\top, q_\bot, \delta)$ where:

- Q is a finite set of *states*,
- $q_\top \in Q$ is the *top* (initial) state,
- $q_\bot \in Q$ is the *bottom* (terminal) state,
- $\delta : Q \times \mathcal{M} \rightharpoonup Q$ is a partial *transition function* ($\mathcal{M}$ = method names).

We write $s \xrightarrow{m} s'$ when $\delta(s, m) = s'$.

For branch-only types, $\mathcal{L}(S)$ is a finite tree rooted at $q_\top$ with all leaves identified as $q_\bot$. By [2], $\mathcal{L}(S)$ forms a lattice under the reachability ordering $s_1 \geq s_2 \iff s_1 \to^* s_2$, with $q_\top = \top$ and $q_\bot = \bot$.

Example 2.3 (FileHandle state space)**.** $\mathcal{L}(S_{\text{FH}})$ has five states:



where $s_0 = \top$ (green) and $s_4 = \bot$ (red). The enabled methods at each state are: $s_0 \mapsto \{open\}$, $s_1 \mapsto \{read, write\}$, $s_2, s_3 \mapsto \{close\}$, $s_4 \mapsto \emptyset$.

3 Syntax of λ_S^0

We formalise the client-side language—the code that *uses* session-typed objects. Object implementations are treated as black boxes conforming to their declared session type.

3.1 BNF Grammar

Definition 3.1 (Terms)**.**

$$
\begin{array}{rcll}
e & ::= & v & \text{(value)} \\
 & | & \textbf{new}\ C() & \text{(allocation)} \\
 & | & v.m() & \text{(method call)} \\
 & | & \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{(sequencing)} \\
v & ::= & x & \text{(variable)} \\
 & | & \textbf{skip} & \text{(unit value)}
\end{array}
$$

where C ranges over class names and m over method names.

Design rationale. The syntax is a strict subset of Featherweight Java [10], minus casts, inheritance, and field access. We retain **let** rather than statement sequencing $(e_1; e_2)$ to make the scoping of variables explicit—this is essential for linearity checking. Methods return **skip** (unit); richer return types belong to λ_S^1.

Remark 3.2 (Method arguments)*.* Methods in λ_S^0 are nullary: $v.m()$ takes no arguments. In practice, Java methods may accept parameters, including objects of session-typed classes. We permit method parameters and return values of *non-session-typed* types (primitives, strings, etc.) without loss of soundness—they do not affect the typing environment Γ. However, *passing a session-typed object as an argument* requires additional machinery not present in λ_S^0–λ_S^6: method effect signatures, linearity enforcement, and a frame rule. See Section 8.4 for a detailed discussion and Section 8.5 for related work.

3.2 Values and Programs

Definition 3.3 (Values)**.** A *value* is either a variable x or **skip**.

Definition 3.4 (Programs)**.** A *program* is a pair $(\mathcal{CT}, e)$ where:

- $\mathcal{CT}$ is a *class table* mapping each class name C to its session type $\mathcal{CT}(C) \in S$, and
- e is a closed term (no free variables).

3.3 Well-Formedness

Definition 3.5 (Well-formed term)**.** A term e is *well-formed* with respect to $\mathcal{CT}$ if:

1. Every **new** $C()$ refers to a class $C \in \mathrm{dom}(\mathcal{CT})$.
2. Every $v.m()$ has v as a variable (not **skip**).
3. Every **let** $x = e_1$ **in** e_2 binds a fresh x not already in scope.

4 Type System

The type system is the core of λ_S^0. It tracks the session state of each object through a *pre/post typing environment*, ensuring that method calls follow the protocol and that every object reaches its terminal state.

4.1 Types and Environments

Definition 4.1 (Types)**.**

$$T \quad ::= \quad \mathbf{Unit} \quad | \quad C$$

where **Unit** is the type of **skip** and C is a class type.

Definition 4.2 (Typing environment)**.** A *typing environment* Γ is a finite map from variables to *session-typed entries*:

$$\Gamma \quad ::= \quad \emptyset \quad | \quad \Gamma,\, x : C \mathbin{@} s$$

where $s \in Q$ is a state of $\mathcal{L}(\mathcal{CT}(C))$. We write $\Gamma(x) = C \mathbin{@} s$ for lookup and $\Gamma[x \mapsto C \mathbin{@} s']$ for update.

4.2 Typing Judgement

The central judgement has the form:

$$\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$$

Read: "Under class table $\mathcal{CT}$ and input environment Γ, expression e has type T and produces output environment Γ'." The *pre*-environment Γ records session states before e executes; the *post*-environment Γ' records states after.

4.3 Typing Rules

We present five rules. Each is annotated with the corresponding location in Bica Reborn's `TypestateChecker.java`.

$$\textsc{T-Skip}\ \frac{}{\mathcal{CT};\ \Gamma \vdash \mathbf{skip} : \mathbf{Unit} \dashv \Gamma}$$

T-Skip. The unit value **skip** has type **Unit** and does not modify the environment. This rule has no direct counterpart in `TypestateChecker`; it corresponds to a no-op between method calls.

$$\textsc{T-Var}\ \frac{(x : C \mathbin{@} s) \in \Gamma}{\mathcal{CT};\ \Gamma \vdash x : C \dashv \Gamma}$$

T-Var. Variable lookup. The environment is unchanged because merely referring to a variable does not advance the protocol. In `TypestateChecker`, this corresponds to the variable-to-state lookup at line 43: `Integer state = currentStates.get(call.variable())`.

$$\text{T-New}\ \frac{C \in \mathrm{dom}(\mathcal{CT}) \qquad x \notin \mathrm{dom}(\Gamma) \qquad q_\top = \mathrm{top}(\mathcal{L}(\mathcal{CT}(C)))}{\mathcal{CT};\ \Gamma \vdash \textbf{new}\ C() : C \dashv \Gamma,\ x : C \mathbin{@} q_\top}$$

T-New. Object allocation introduces a fresh variable at the top state of its session type's state space. In `TypestateChecker`, this corresponds to `initialStates.put(v, stateSpace.top())` at line 100.

$$\text{T-Call}\ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s'}{\mathcal{CT};\ \Gamma \vdash x.m() : \textbf{Unit} \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

T-Call. The critical rule: a method call $x.m()$ is well-typed only if method m is enabled in the current state s of x, i.e., the transition $\delta(s, m) = s'$ exists. The output environment advances x to state s'.

In `TypestateChecker`, this corresponds to lines 52–65:

```
var next = stateSpace.step(state, call.method());
if (next.isEmpty()) { /* error:  method not enabled */ }
else { currentStates.put(call.variable(), next.getAsInt()); }
```

The premise $\delta(s, m) = s'$ corresponds to the check `next.isEmpty()`, and the environment update corresponds to `currentStates.put(...)`. If m is not enabled ($\delta(s, m)$ undefined), the term is *not well-typed*, corresponding to `TypestateChecker` reporting a `TypestateError`.

$$\text{T-Let}\ \frac{\mathcal{CT};\ \Gamma \vdash e_1 : T_1 \dashv \Gamma_1 \qquad \mathcal{CT};\ \Gamma_1, x : T_1' \vdash e_2 : T_2 \dashv \Gamma_2}{\mathcal{CT};\ \Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : T_2 \dashv \Gamma_2}$$

T-Let. Sequences two expressions, threading the environment: the output of e_1 becomes the input of e_2. The variable x is bound to the result of e_1 in the scope of e_2. Here $T_1' = T_1$ when T_1 is **Unit**, and $T_1' = C \mathbin{@} q_\top$ when $e_1 = \textbf{new}\ C()$ (the binding carries session state from the T-New rule).

In `TypestateChecker`, this corresponds to the sequential loop over calls at lines 42–66: each iteration processes one call and carries the updated `currentStates` map to the next iteration.

4.4 Session Completion

Definition 4.3 (Session completion)**.** A typing derivation $\mathcal{CT}; \emptyset \vdash e : T \dashv \Gamma'$ satisfies *session completion* if for every $(x : C \mathbin{@} s) \in \Gamma'$ we have $s = q_\perp$, where $q_\perp = \mathrm{bot}(\mathcal{L}(\mathcal{CT}(C)))$.

In `TypestateChecker`, session completion corresponds to the `checkEnd` flag (lines 68–84): when enabled, the checker verifies that all tracked variables are in the bottom state.

```
if (finalState != stateSpace.bottom()) { /* error */ }   (line 72)
```

A well-typed λ_S^0 program is a pair $(\mathcal{CT}, e)$ such that $\mathcal{CT}; \emptyset \vdash e : T \dashv \Gamma'$ is derivable *and* session completion holds.

5 Operational Semantics

We give a small-step operational semantics in the style of Wright–Felleisen [15]. Reduction operates on *configurations* that track the runtime heap and session states.

5.1 Runtime Entities

Definition 5.1 (Locations)**.** $\ell \in \mathit{Loc}$ ranges over an infinite set of heap locations.

Definition 5.2 (Heap)**.** A *heap* $H : \mathit{Loc} \rightharpoonup \mathit{ClassName}$ maps locations to their class.

Definition 5.3 (Session store)**.** A *session store* $\sigma : \mathit{Loc} \rightharpoonup Q$ maps locations to their current session state.

Definition 5.4 (Configuration)**.** A *configuration* is a triple $\langle H,\ \sigma,\ e\rangle$ where H is a heap, σ is a session store, and e is a (runtime) expression. Runtime expressions extend the source syntax with locations:

$$v \ ::= \ x \ \mid \ \textbf{skip} \ \mid \ \ell$$

5.2 Reduction Rules

Definition 5.5 (Reduction)**.** The reduction relation $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H',\ \sigma',\ e'\rangle$ is defined by the following four rules:

$$\textsc{R-New}\ \frac{\ell \notin \mathrm{dom}(H) \qquad q_\top = \mathrm{top}(\mathcal{L}(\mathcal{CT}(C)))}{\langle H,\ \sigma,\ \textbf{new}\ C()\rangle \longrightarrow \langle H[\ell \mapsto C],\ \sigma[\ell \mapsto q_\top],\ \ell\rangle}$$

R-New. Allocation creates a fresh location ℓ, records its class in the heap, and initialises its session state to $q_\top$.

$$\textsc{R-Call}\ \frac{H(\ell) = C \qquad \sigma(\ell) = s \qquad \delta(s, m) = s'}{\langle H,\ \sigma,\ \ell.m()\rangle \longrightarrow \langle H,\ \sigma[\ell \mapsto s'],\ \textbf{skip}\rangle}$$

R-Call. A method call on location ℓ advances the session state from s to s' via the transition $\delta(s, m) = s'$. The call reduces to **skip**. If no transition exists ($\delta(s, m)$ undefined), the configuration is *stuck*—this models a `ProtocolViolationException`.

$$\textsc{R-Let-Ctx}\ \frac{\langle H,\ \sigma,\ e_1\rangle \longrightarrow \langle H',\ \sigma',\ e_1'\rangle}{\langle H,\ \sigma,\ \textbf{let}\ x = e_1\ \textbf{in}\ e_2\rangle \longrightarrow \langle H',\ \sigma',\ \textbf{let}\ x = e_1'\ \textbf{in}\ e_2\rangle}$$

R-Let-Ctx. Congruence rule: reduce the binding expression e_1 first (left-to-right, call-by-value).

$$\textsc{R-Let-Val}\ \frac{v \text{ is a value}}{\langle H,\ \sigma,\ \textbf{let}\ x = v\ \textbf{in}\ e_2\rangle \longrightarrow \langle H,\ \sigma,\ e_2[v/x]\rangle}$$

R-Let-Val. When the binding position holds a value, substitute it into the body. The heap and session store are unchanged.

5.3 Stuck Configurations

Definition 5.6 (Stuck)**.** A configuration $\langle H,\ \sigma,\ e\rangle$ is *stuck* if e is not a value and no reduction rule applies. The only way a well-formed configuration can be stuck is when $e = \ell.m()$ with $\sigma(\ell) = s$ and $\delta(s, m)$ undefined.

A stuck configuration corresponds to a runtime protocol violation—the client attempted a method call that the session type does not permit in the current state. In the Bica Reborn implementation, this is caught by `TypestateChecker` and reported as a `TypestateError` (lines 55–62).

6 Type Soundness

We prove that the λ_S^0 type system is sound: well-typed programs do not get stuck and drive all objects to their terminal state. The proof follows the standard syntactic approach of [15].

6.1 Configuration Typing

To state preservation, we extend typing to configurations.

Definition 6.1 (Well-typed configuration)**.** A configuration $\langle H,\ \sigma,\ e\rangle$ is *well-typed*, written $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$, if there exists a typing environment Γ such that:

1. $\mathcal{CT}; \Gamma \vdash e : T \dashv \Gamma'$,
2. For every $\ell \in \mathrm{dom}(H)$: $H(\ell) = C$ and $\sigma(\ell) = s$ implies $(\ell : C \mathbin{@} s) \in \Gamma$, and
3. For every $(x : C \mathbin{@} s) \in \Gamma$ where $x = \ell$: s is a state of $\mathcal{L}(\mathcal{CT}(C))$.

6.2 Preservation

Theorem 6.2 (Preservation)**.** *If $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$ and $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H_1,\ \sigma_1,\ e_1\rangle$, then $\mathcal{CT} \vdash \langle H_1,\ \sigma_1,\ e_1\rangle : T \dashv \Gamma'$.*

Proof sketch. By case analysis on the reduction rule applied.

Case R-New. $e = \textbf{new}\ C()$ reduces to ℓ with $H_1 = H[\ell \mapsto C]$ and $\sigma_1 = \sigma[\ell \mapsto q_\top]$. By T-New, the pre-derivation gives $\Gamma' = \Gamma, x : C \mathbin{@} q_\top$. After reduction, ℓ has class C and state $q_\top$ in the heap, so the location ℓ satisfies $(\ell : C \mathbin{@} q_\top) \in \Gamma'$. The type of ℓ is C, matching T. The output environment Γ' is unchanged.

Case R-Call. $e = \ell.m()$ with $\sigma(\ell) = s$ and $\delta(s, m) = s'$. By T-Call, $\Gamma(\ell) = C \mathbin{@} s$ and $\Gamma' = \Gamma[\ell \mapsto C \mathbin{@} s']$. After reduction: $\sigma_1(\ell) = s'$ and $e_1 = \textbf{skip}$. By T-Skip, $\mathcal{CT}; \Gamma' \vdash \textbf{skip} : \textbf{Unit} \dashv \Gamma'$. The heap is unchanged. The session store matches the output environment: $\sigma_1(\ell) = s'$ and $(\ell : C \mathbin{@} s') \in \Gamma'$.

Case R-Let-Ctx. $e = \textbf{let}\ x = e_1\ \textbf{in}\ e_2$ and $e_1 \longrightarrow e_1'$. By T-Let, $\mathcal{CT}; \Gamma \vdash e_1 : T_1 \dashv \Gamma_1$. By induction hypothesis on the sub-derivation for e_1, $\mathcal{CT} \vdash \langle H_1,\ \sigma_1,\ e_1'\rangle : T_1 \dashv \Gamma_1$. Reapplying T-Let with the unchanged derivation for e_2 gives $\mathcal{CT} \vdash \langle H_1,\ \sigma_1,\ \textbf{let}\ x = e_1'\ \textbf{in}\ e_2\rangle : T_2 \dashv \Gamma_2$.

Case R-Let-Val. $e = \textbf{let}\ x = v\ \textbf{in}\ e_2$ reduces to $e_2[v/x]$. By T-Let, $\mathcal{CT}; \Gamma \vdash v : T_1 \dashv \Gamma_1$ and $\mathcal{CT}; \Gamma_1, x : T_1' \vdash e_2 : T_2 \dashv \Gamma_2$. The substitution $e_2[v/x]$ replaces x with v in e_2. Since v has the same type as x (both T_1, or a location of class C at the same state), and the session state of v in Γ_1 matches the binding $x : T_1'$, by the substitution lemma $\mathcal{CT}; \Gamma_1 \vdash e_2[v/x] : T_2 \dashv \Gamma_2$. The heap and session store are unchanged. □

Lemma 6.3 (Substitution)**.** *If $\mathcal{CT}; \Gamma, x : C \mathbin{@} s \vdash e : T \dashv \Gamma'$ and v is a value with $\Gamma(v) = C \mathbin{@} s$, then $\mathcal{CT}; \Gamma \vdash e[v/x] : T \dashv \Gamma'[v/x]$.*

Proof sketch. By structural induction on the typing derivation. Each rule either does not mention x (trivial) or replaces x with v while preserving the session state. □

6.3 Progress

Theorem 6.4 (Progress)**.** *If $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$, then either:*

1. *e is a value, or*
2. *there exist H', σ', e' such that $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H',\ \sigma',\ e'\rangle$.*

Proof sketch. By case analysis on the form of e.

Case $e = \textbf{skip}$. Already a value.

Case $e = x$. If x is a location ℓ, it is a value. If x is a source variable in a well-typed closed term, it must be bound by an enclosing **let**, so this case does not arise for closed configurations.

Case $e = \textbf{new}\ C()$. Since $C \in \mathrm{dom}(\mathcal{CT})$ (by T-New), we can always choose a fresh $\ell \notin \mathrm{dom}(H)$ (locations are infinite). R-New applies.

Case $e = \ell.m()$. By well-typedness, $\Gamma(\ell) = C \mathbin{@} s$ and $\delta(s, m) = s'$ for some s' (the premise of T-Call). Since $\sigma(\ell) = s$ (by configuration typing), R-Call applies. *This is the key case:* the T-Call typing rule guarantees that the transition exists, so the configuration is never stuck.

Case $e = \textbf{let}\ x = e_1\ \textbf{in}\ e_2$. If e_1 is a value, R-Let-Val applies. Otherwise, by induction hypothesis on e_1, it can reduce, and R-Let-Ctx applies. □

6.4 Session Completion

Corollary 6.5 (Session Completion)**.** *If program $(\mathcal{CT}, e)$ is well-typed with session completion (Definition 4.3) and $\langle \emptyset,\ \emptyset,\ e\rangle \longrightarrow^* \langle H,\ \sigma,\ v\rangle$ (i.e., e reduces to a value), then $\sigma(\ell) = q_\perp$ for every $\ell \in \mathrm{dom}(H)$.*

Proof sketch. By iterated application of Theorem 6.2: the well-typed configuration at each step has the same output environment Γ'. When the expression reduces to a value, the output environment is the post-environment of the entire derivation. By Definition 4.3, every entry in Γ' has state $q_\perp$. By configuration typing (Definition 6.1), $\sigma(\ell)$ matches the state in Γ' for each location ℓ, so $\sigma(\ell) = q_\perp$. □

Remark 6.6 (Termination). λ_S^0 programs always terminate because the session type is a finite tree and each method call strictly descends in the tree (consuming one branch constructor). There are no loops or recursive calls in the term language. Thus the reduction sequence $\langle \emptyset,\ \emptyset,\ e\rangle \longrightarrow^* \langle H,\ \sigma,\ v\rangle$ is always finite, and Corollary 6.5 applies to all well-typed programs.

7 Worked Examples

We demonstrate the type system on the `FileHandle` protocol from Examples 2.2 and 2.3. Let $\mathcal{CT}(\texttt{FileHandle}) = S_{\mathrm{FH}}$.

7.1 Correct Client

Consider the client program:

$$e_1 = \textbf{let}\ f = \textbf{new}\ \texttt{FileHandle}()\ \textbf{in let}\ _ = f.\mathit{open}()\ \textbf{in let}\ _ = f.\mathit{read}()\ \textbf{in}\ f.\mathit{close}()$$

The typing derivation proceeds as follows. We write Γ_i for the environment after step i, and use $F = \texttt{FileHandle}$ for brevity.

1. By T-New: $\mathcal{CT}; \emptyset \vdash \textbf{new}\ F() : F \dashv \{f : F \mathbin{@} s_0\}$
2. Let $\Gamma_0 = \{f : F \mathbin{@} s_0\}$. By T-Call: $\mathcal{CT}; \Gamma_0 \vdash f.\mathit{open}() : \textbf{Unit} \dashv \{f : F \mathbin{@} s_1\}$ since $\delta(s_0, \mathit{open}) = s_1$.

3. Let $\Gamma_1 = \{f : F \mathbin{@} s_1\}$. By T-Call: $\mathcal{CT}; \Gamma_1 \vdash f.\mathit{read}() : \mathbf{Unit} \dashv \{f : F \mathbin{@} s_2\}$ since $\delta(s_1, \mathit{read}) = s_2$.
4. Let $\Gamma_2 = \{f : F \mathbin{@} s_2\}$. By T-Call: $\mathcal{CT}; \Gamma_2 \vdash f.\mathit{close}() : \mathbf{Unit} \dashv \{f : F \mathbin{@} s_4\}$ since $\delta(s_2, \mathit{close}) = s_4$.
5. Final environment: $\Gamma' = \{f : F \mathbin{@} s_4\}$. Since $s_4 = q_\perp$, session completion holds.

The derivation traces the path $s_0 \xrightarrow{\mathit{open}} s_1 \xrightarrow{\mathit{read}} s_2 \xrightarrow{\mathit{close}} s_4$ in the Hasse diagram—a root-to-leaf path.

7.2 Rejected Client

Consider the ill-typed program:

$$e_2 = \mathbf{let}\ f = \mathbf{new}\ \texttt{FileHandle}()\ \mathbf{in}\ f.\mathit{read}()$$

The typing derivation fails:

1. By T-New: $\mathcal{CT}; \emptyset \vdash \mathbf{new}\ F() : F \dashv \{f : F \mathbin{@} s_0\}$.
2. Attempt T-Call with $\Gamma_0 = \{f : F \mathbin{@} s_0\}$: need $\delta(s_0, \mathit{read}) = s'$ for some s'. But $\mathit{read} \notin \mathrm{enabled}(s_0) = \{\mathit{open}\}$, so $\delta(s_0, \mathit{read})$ is undefined.
3. T-Call's premise is not satisfied. *The program is not well-typed.*

In Bica Reborn, this corresponds to `TypestateChecker` reporting: `Method 'read' is not enabled on 'f' in state 0 (enabled: [open])`.

7.3 Incomplete Client

Consider:

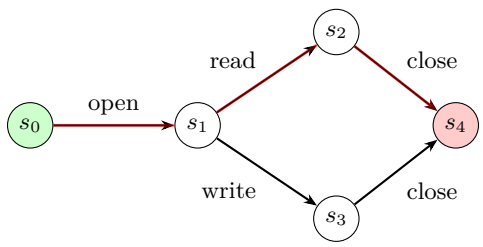
$$e_3 = \mathbf{let}\ f = \mathbf{new}\ \texttt{FileHandle}()\ \mathbf{in}\ f.\mathit{open}()$$

The typing derivation succeeds for each individual step:

1. By T-New: $\Gamma_0 = \{f : F \mathbin{@} s_0\}$.
2. By T-Call: $\Gamma' = \{f : F \mathbin{@} s_1\}$, since $\delta(s_0, \mathit{open}) = s_1$.

But session completion *fails*: $\Gamma'(f) = F \mathbin{@} s_1$ and $s_1 \neq s_4 = q_\perp$. The object has not reached its terminal state. In Bica Reborn, this is caught by the `checkEnd` logic (line 72): `Variable 'f' is not in terminal state at end of method`.

7.4 Correspondence with the Hasse Diagram



The red path shows the derivation of e_1 (correct client): $s_0 \to s_1 \to s_2 \to s_4$. Every well-typed λ_S^0 program traces exactly one root-to-leaf path in the Hasse diagram. The rejected client e_2 would attempt to jump from s_0 to a node reachable only via *read*, which is not a child of s_0. The incomplete client e_3 stops at s_1, an internal node.

8 Discussion

8.1 Correspondence with Bica Reborn TypestateChecker

Table 1 summarises the correspondence between each formal rule and its implementation in `TypestateChecker.java`.

Table 1: Correspondence between λ_S^0 rules and Bica Reborn implementation.

Rule	**Implementation**	**Lines**
T-New	`initialStates.put(v, stateSpace.top())`	99–101
T-Var	`currentStates.get(call.variable())`	43
T-Call	`stateSpace.step(state, call.method())`	52–65
T-Let	`for (MethodCall call : calls)` loop	42–66
T-Skip	(implicit: no-op between calls)	—
Session completion	`if (finalState != stateSpace.bottom())`	69–84
R-Call stuck	`TypestateError` / `ProtocolViolationException`	55–62

The `TypestateChecker.check()` method (lines 37–87) implements the *flat-call mode* of typestate checking: it takes a `StateSpace` and a `List<MethodCall>` and simulates the typing derivation as a fold over the call sequence, threading a map `currentStates` : Variable → State that corresponds precisely to our typing environment Γ.

Note that `TypestateChecker` also supports selection auto-advance (lines 138–169) and call-tree checking with `SelectionSwitch` nodes (lines 119–277). These features correspond to λ_S^2 (conditionals) and are outside the scope of λ_S^0.

8.2 Roadmap to λ_S^1–λ_S^7

λ_S^0 is the base layer of a stratified calculus family. Each subsequent layer adds one feature:

Layer	**Feature**	**Session type addition**	**Step**
λ_S^0	Straight-line calls	Branch, End	200a
λ_S^1	Return values	(unchanged)	200d
λ_S^2	Conditionals (switch/if)	Select ($\oplus$)	200e
λ_S^3	Loops	Recursion ($\mu X.S$)	200f
λ_S^4	Inheritance	Subtyping	200g
λ_S^5	Generics	Parametric polymorphism	200h
λ_S^6	Concurrency	Parallel ($\|$)	200i
λ_S^7	Session-typed arguments	Linearity, effects	200j
λ_S^8	Distribution	Channel endpoints	200k

Each layer extends the syntax, adds typing rules, extends the operational semantics, and reproves soundness. The key insight is that each extension is *conservative*: λ_S^{k-1} programs remain well-typed in λ_S^k.

8.3 Limitations

λ_S^0 is intentionally minimal:

- No method return values—methods return **skip**.
- No method arguments of session type (see Section 8.4).
- No control flow—the client cannot branch on method outcomes.
- No aliasing—each object is used through a single variable.
- No recursion in the term language or in session types.
- Object implementations are black boxes—we do not formalise the server side.

These restrictions make the type system and proofs straightforward, but they mean λ_S^0 can only typecheck the simplest clients. The layered approach ensures that each restriction is lifted in a controlled, formally verified manner.

Methods may freely accept and return values of *non-session-typed* types (integers, strings, data objects) without affecting the formalization: such values do not appear in Γ and do not interact with the session discipline. The restriction applies only when a method parameter or return value is itself a session-typed object whose protocol state must be tracked.

8.4 Session-Typed Arguments

The most significant gap in the λ_S^0–λ_S^6 hierarchy is the absence of *session-typed method arguments*: the case where a method call $x.m(y)$ passes an object y whose class has a `@Session` annotation. This gap is deliberate—it requires three pieces of machinery that interact with recursion (λ_S^3), concurrency (λ_S^6), and each other:

1. Method effect signatures. Each method must declare its effect on every session-typed parameter as a pre/post state pair. Formally, a method signature becomes:

$$msig(C, m) \;=\; (y_1 : D_1 \mathbin{@} r_1 \to r_1',\, \ldots,\, y_k : D_k \mathbin{@} r_k \to r_k') \to R$$

The T-Call rule must then update Γ for every argument:

$$\Gamma' = \Gamma[x \mapsto C \mathbin{@} s',\; a_1 \mapsto D_1 \mathbin{@} r_1',\; \ldots,\; a_k \mapsto D_k \mathbin{@} r_k']$$

2. Linearity. If the caller passes y to $x.m(y)$, it must not use y concurrently or assume y's state is unchanged after the call. This requires a linear or affine type discipline on Γ: the entry for y is *consumed* by the call and replaced with its post-state r'. Without linearity, aliasing permits double-use violations that break session safety.

3. Frame rule. The caller must know that the method does *not* affect variables outside its declared parameters. This is the session-type analogue of separation logic's frame rule: Γ_{frame} (the environment entries not mentioned in the method signature) is preserved across the call.

These three concerns interact with the features in the hierarchy: recursive types (λ_S^3) may create infinite-state protocols on arguments; concurrency (λ_S^6) introduces shared access where linearity becomes critical. We therefore place session-typed arguments as λ_S^7, *after* recursion and concurrency have been formalized, so that the interaction can be handled in full generality.

8.5 Related Work on Session-Typed Arguments

Several systems address the problem of passing session-typed objects across method boundaries:

- **Mungo** [11]: tracks typestate for Java objects, including through method parameters. Linearity is enforced by forbidding aliasing of protocol-tracked references; passing an object drains the caller's reference (move semantics).
- **Modular Session Types** [7, 8]: the original BICA system and its full journal version formalise a *remaining-type split* at method boundaries. The method signature declares which prefix of the session type it consumes; the caller retains the continuation type.
- **Vault/Fugue** [5, 6]: the *adoption/focus* mechanism reconciles typestate with aliasing. Adoption transfers ownership to a container; focus temporarily reclaims linear access for a method call, with pre/post state declarations.
- **Plaid** [1]: access permissions (unique, shared, immutable) with pre/post annotations (`unique` $\gg$ `unique`) control how objects flow across method boundaries. Naden et al. [12] formalise a type system for borrowing permissions.
- **Borrowing from Session Types** [4]: the most recent approach, using ordered linear typing with a borrow operation. Methods can borrow a session-typed channel, consume part of its protocol, and return it—avoiding the awkward continuation-passing style that strict linearity imposes.

- **Gay, Vasconcelos, and Ravara** [14]: formalise session environments for a multithreaded functional language, threading linear session channels through function application.

A full treatment for λ_S^7 will draw on these approaches, adapting their mechanisms to the lattice-structured state spaces that distinguish our framework.

9 Conclusion

We have presented λ_S^0, a formal calculus for straight-line typestate checking of session-typed objects. The calculus has five typing rules (T-Skip, T-Var, T-New, T-Call, T-Let), four reduction rules (R-New, R-Call, R-Let-Ctx, R-Let-Val), and a type soundness result (preservation, progress, session completion). Every rule has a documented correspondence to the Bica Reborn `TypestateChecker` implementation.

λ_S^0 retroactively grounds the existing implementation in a formal specification, fulfilling the formalisation-first policy. As the base layer of the λ_S^k hierarchy, it provides a foundation on which richer features—return values, conditionals, loops, inheritance, concurrency—can be added incrementally with full formal justification.

Future work. Step 200b mechanises the λ_S^0 soundness proof in Lean 4. Step 200c adds automated alignment tests that verify the Bica Reborn implementation against the formal rules. Step 200d extends to λ_S^1 with method return values (non-session-typed). Subsequent layers add conditionals (λ_S^2), loops (λ_S^3), inheritance (λ_S^4), generics (λ_S^5), and concurrency (λ_S^6). The treatment of *session-typed method arguments* (λ_S^7, Section 8.4) is deliberately deferred until after concurrency, since linearity, method effect signatures, and the frame rule interact with recursive and parallel session types.

References

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. Typestate-oriented programming. In *OOPSLA 2009*, pages 1015–1022. ACM, 2009.

[2] Alexandre Zua Caldeira. Session types as algebraic reticulates: The lattice theorem (step 5), 2026. Submitted to ICE 2026.

[3] Alexandre Zua Caldeira. State-space construction for binary session types (step 1), 2026. arXiv preprint.

[4] Luca Ciccone, Ornela Dardha, and Simon J. Gay. Borrowing from session types. *Proc. ACM Program. Lang.*, 9(OOPSLA), 2025.

[5] Robert DeLine and Manuel Fähndrich. Adoption and focus: Practical linear types for imperative programming. In *PLDI 2002*, pages 13–24. ACM, 2002.

[6] Robert DeLine and Manuel Fähndrich. Typestates for objects. In *ECOOP 2004*, volume 3086 of *LNCS*, pages 465–490. Springer, 2004.

[7] Simon Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *POPL 2010*, pages 299–312. ACM, 2010.

[8] Simon J. Gay, Nils Gesbert, António Ravara, and Vasco T. Vasconcelos. Modular session types for objects. *Logical Methods in Computer Science*, 11(4:12):1–76, 2015.

[9] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP 1998*, volume 1381 of *LNCS*, pages 122–138. Springer, 1998.

[10] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001.

[11] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo. In *PPDP 2016*, pages 146–159. ACM, 2016.

[12] Karl Naden, Robert Bocchino, Jonathan Aldrich, and Kevin Myers. A type system for borrowing permissions. pages 557–570, 2012.

[13] Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, 1986.

[14] Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006.

[15] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.